

```

for(int i = 2; i < groupCounter + 1; i++){
    cout << groupReps[i-1] << " " << groupReps[i] << endl;}
return 0;}

```

Problem 12:

Input

The input consists of several test cases. Each test case starts with a line with four non-negative integers, $1 \leq n \leq 10\,000$, $0 \leq m \leq 30\,000$, $1 \leq q \leq 100$ and $0 \leq s < n$, separated by single spaces, where n is the numbers of nodes in the graph, m the number of edges, q the number of queries and s the index of the starting node. Nodes are numbered from 0 to $n - 1$. Then follow m lines, each line consisting of three (space-separated) integers u , v and w indicating that there is an edge from u to v in the graph with weight $0 \leq w \leq 1000$. Then follow q lines of queries, each consisting of a single non-negative integer, asking for the minimum distance from node s to the node number given on the query line.

Input will be terminated by a line containing four zeros, this line should *not* be processed.

Output

For each query, output a single line containing the minimum distance from node s to the node specified in the query, or the word "Impossible" if there is no path from s to that node. For clarity, the sample output has a blank line between the output for different cases.

Solution 12:

```

vector<int> dijkstra(vector<vector<pair<int, int>>>& G, int s){
    int Size = G.size();
    priority_queue<pair<int, int>, vector<pair<int, int>>, greater<pair<int, int>>> pq;
    //distances start at inf
    vector<int> d(Size, INT_MAX);
    d[s] = 0;
    pq.emplace(0, s);
    while(!pq.empty()){
        auto t = pq.top();
        pq.pop();
        int dist = t.first;
        int u = t.second;
        if(dist > d[u]){
            continue;
        }
        for(auto &p : G[u]){
            int v = p.first;
            int w = p.second;
            if(d[u] + w < d[v]){
                d[v] = d[u] + w;
                pq.emplace(d[v], v);
            }
        }
    }
    return d;
}

int main(){
    while(true) {
        int n, m, q, s;
        cin >> n >> m >> q >> s;
        if(n == 0 && m == 0 && q == 0 && s == 0){
            break;
        }
        //should store <distance, node>
        vector<vector<pair<int, int>>> G (n);
        for(int i = 0; i < m; i++){
            int u, v, w;

```